

VC Generation

Overview

Goal

VC generation turns a TAC-like program into an SMT query.

$$\text{sat}(\text{VC}(P, A)) \Leftrightarrow \text{there exists an execution reaching failure of } A$$

Tiny TAC (`ttac`) keeps the semantic core visible:

- explicit basic blocks and terminators
- named Boolean branch and assertion conditions
- scalar definitions, havoc, assume, assert
- maps and map updates
- phi nodes at joins

What The Encoder Consumes

A ttac program gives the VC generator:

- a control-flow graph
- commands in each block
- symbol sorts
- scalar and map expressions
- the assertion-failure query

The encoder produces an SMT formula whose models are bug witnesses.

$$VC(P, A) = \text{Path} \wedge \text{Assumes} \wedge \neg \text{Assert}$$

Unsat means no modeled assertion-failure execution exists.

VCGen Styles

- **SeaHorn-style**: explicit block reachability variables.
- **Boogie-style**: backward safety equations.
- **SeaBMC-style**: control-dependence compiled into data-dependence.

The encodings differ mainly in how they represent path semantics and joins.

Running Example Shape

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto left else right

left:
  a_left := x + 1
  goto join

right:
  a_right := y + 1
  goto join
```

Running Example: Join And Exit

```
join:
  a := phi [left: a_left, right: a_right]
  ok := a > 0
  goto exit

exit:
  assume true
  assert ok
  halt
```

This diamond exercises branch conditions, joins, phi nodes, definitions, and the final failure query.

Reading The VC

The query orientation is:

$\text{SAT} \Rightarrow$ a bug witness exists

$\text{UNSAT} \Rightarrow$ the assertion is proved in the modeled semantics

The main design choice is how to represent path semantics:

- explicit reachability variables
- recursive safety equations
- data-flow gates and ITEs

Tiny TAC

Syntax And Semantics

Tiny TAC

Tiny TAC (`ttac`) is the small language used in the VCGen notes.

It keeps only the semantic core needed for examples:

- typed registers: `bool`, `int`, `bytemap`
- assignments, `havoc`, `assume`, `assert`
- basic blocks with explicit terminators
- phi nodes at joins

The goal is not to document `ctac` syntax. The goal is to make the VCGen algorithms precise.

Types And Expressions

Registers have one of three types:

$$\tau ::= \text{bool} \mid \text{int} \mid \text{bytemap}$$

The SMT model is intentionally direct:

$$\text{int} \mapsto \text{Int}$$

$$\text{bytemap} \mapsto \text{Int} \rightarrow \text{Int}$$

Core integer expressions:

$$i ::= n \mid x \mid i + i \mid i - i \mid i * i \mid \frac{i}{i}$$

Boolean expressions:

$$b ::= \text{true} \mid \text{false} \mid c \mid i < i \mid i \leq i \mid b \wedge b$$

ite is an expression operator, not a structured statement.

Maps

Loads and stores are expressions over bytemaps.

```
M := havoc  
i := havoc  
v := havoc  
x := M[i]  
M2 := M[i := v]
```

Semantically, M_2 agrees with M at every address except i , where it stores v .

Commands

Command forms:

```
x := e
x := havoc
x := phi [B1: x1, B2: x2]
assume b
assert c
```

`:=` is assignment. Equality is a separate expression/operator.

Example:

```
x := havoc
y := havoc
z := havoc
c := x < y
assume y < z
assert c
```

`assume` may use an expression. `assert` uses a named bool register.

Blocks And Control

Unstructured basic blocks. Branches are terminators.

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto ok else bad
```

```
ok:
  assert c
  goto exit
```

```
bad:
  assume not c
  halt
```

```
exit:
  halt
```

Phi Nodes

Phi nodes select by predecessor block.

```
L:
  x1 := 1
  goto J

R:
  xr := 2
  goto J

J:
  x := phi [L: x1, R: xr]
  goto exit
```

- Enter from L: $x := x_l$
- Enter from R: $x := x_r$

Semantic Contract

A ttac execution follows the expected operational semantics:

- assignment updates the destination register
- havoc chooses an arbitrary value of the destination type
- assume filters executions
- assert fails when its named condition is false
- terminators choose the next block

VC generation is bug-oriented:

$\text{VCGen}(P)$ is satisfiable $\Leftrightarrow P$ has an assertion-failure execution

VC Generation

Well-Formed Programs

Why Preconditions Exist

The grammar accepts more programs than the encoders should consume.

Well-formedness records the shape expected by each VCGen style:

- CFG shape: entry, exit, paths
- definition discipline: SSA or DSA
- assertion discipline: single final failure query
- join discipline: phi nodes or edge-local dynamic definitions

CFG Vocabulary

A CFG has:

- one node per basic block
- edge $b \rightarrow b'$ when b can transfer to b'
- successors from outgoing edges
- predecessors from incoming edges

```
entry:
  c := havoc
  if c goto left else right

left:
  goto join

right:
  goto join
```

SESE

A program is **single-entry single-exit** when:

- entry and exit are distinguished and distinct
- execution starts only at entry
- normal completed executions end at exit
- every block lies on an entry-to-exit path

SESE = no unreachable blocks, no dead regions, one normal exit

SSA

Static single assignment:

- every register is assigned at most once
- phi assignments are a prefix of the join block
- every phi has one incoming value per predecessor

```
left:
  x_left := 1
  goto join

right:
  x_right := 2
  goto join

join:
  x := phi [left: x_left, right: x_right]
```

DSA

Dynamic single assignment pushes the merge onto incoming edges.

```
left:
  x_left := 1
  x := x_left
  goto join

right:
  x_right := 2
  x := x_right
  goto join

join:
  goto exit
```

The two assignments to x are allowed because they occur in sibling predecessors, immediately before the join.

SSA To DSA

Before:

```
join:  
  x := phi [p1: v1, p2: v2]  
  goto exit
```

After:

```
p1:  
  x := v1  
  goto join  
  
p2:  
  x := v2  
  goto join
```

DSA can be read as placing the merge assignment on the incoming CFG edge.

SASA

Single-assume single-assert form puts the query at the exit:

```
exit:  
  assume pre  
  assert post  
  halt
```

- exactly one assume
- exactly one assert
- pre accumulates path assumptions
- post summarizes assertion obligations

Safety Query

For a SASA program, safety is:

$$\forall \xi. \text{entry-to-exit}(\xi) \Rightarrow (\text{pre}(\xi) \Rightarrow \text{post}(\xi))$$

The VC asks for the negation:

$$\exists \xi. \text{entry-to-exit}(\xi) \wedge \text{pre}(\xi) \wedge \neg \text{post}(\xi)$$

VCGen

SeaHorn Style

Idea

SeaHorn-style VCGen keeps control flow explicit.

For each block i , introduce a Boolean reachability variable:

$$bb_i$$

Then constrain the selected blocks so they describe an entry-to-exit path.

Formula Shape

$$\text{VCGen_SeaHorn}(P) = \text{CFG} \wedge \text{DEF} \wedge \text{JUMPS} \wedge \text{PHI} \wedge \text{ERROR}$$

- CFG: selected block variables form a path.
- DEF: assignments hold when their block is selected.
- JUMPS: selected branch edges satisfy branch conditions.
- PHI: selected incoming edges choose phi values.
- ERROR: the selected exit violates the assertion.

CFG Constraints

$$\begin{aligned} \text{CFG} &= \text{bb}_{\text{entry}} \\ &\wedge \forall i \neq \text{entry}. \text{bb}_i \Rightarrow \bigvee_{j \in \text{pred}(i)} \text{bb}_j \\ &\wedge \text{bb}_{\text{exit}} \end{aligned}$$

Concrete encoders may add edge variables, forward constraints, dominator constraints, or at-most-one constraints. The semantic role is the same: describe feasible control.

Running Diamond

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto left else right

left:
  a_left := x + 1
  goto join

right:
  a_right := y + 1
  goto join
```

Join And Error

```
join:
  a := phi [left: a_left, right: a_right]
  ok := a > 0
  goto exit

exit:
  assume true
  assert ok
  halt
```

Encoded Pieces

$$\begin{aligned}\mathbf{DEF} = & (\mathbf{bb}_{\text{entry}} \Rightarrow c = (x < y)) \\ & \wedge (\mathbf{bb}_{\text{left}} \Rightarrow a_{\text{left}} = x + 1) \\ & \wedge (\mathbf{bb}_{\text{right}} \Rightarrow a_{\text{right}} = y + 1) \\ & \wedge (\mathbf{bb}_{\text{join}} \Rightarrow \text{ok} = (a > 0))\end{aligned}$$

$$\begin{aligned}\mathbf{JUMPS} = & ((\mathbf{bb}_{\text{entry}} \wedge \mathbf{bb}_{\text{left}}) \Rightarrow c) \\ & \wedge ((\mathbf{bb}_{\text{entry}} \wedge \mathbf{bb}_{\text{right}}) \Rightarrow \neg c)\end{aligned}$$

$$\begin{aligned}\mathbf{PHI} = & ((\mathbf{bb}_{\text{left}} \wedge \mathbf{bb}_{\text{join}}) \Rightarrow a = a_{\text{left}}) \\ & \wedge ((\mathbf{bb}_{\text{right}} \wedge \mathbf{bb}_{\text{join}}) \Rightarrow a = a_{\text{right}})\end{aligned}$$

Error Condition

For the SASA exit block:

```
exit:  
  assume pre  
  assert post  
  halt
```

emit:

$$\mathbf{ERROR} = \text{bb}_{\text{exit}} \Rightarrow \text{pre} \wedge \neg \text{post}$$

Together with `bb_exit`, this asks for a complete failing execution.

Optimization: Unguarded DEF

Static definitions may be exposed as top-level equations:

Guarded:

$$bb_i \Rightarrow x = y + 1$$

Unguarded:

$$x = y + 1$$

The unguarded form helps substitution and algebraic simplification, but only when the definition and its side conditions are globally safe.

Optimization: Phi As ITE

Edge implications:

$$(bb_i \wedge bb_j) \Rightarrow x = x_i$$

$$(bb_k \wedge bb_j) \Rightarrow x = x_k$$

One defining equation:

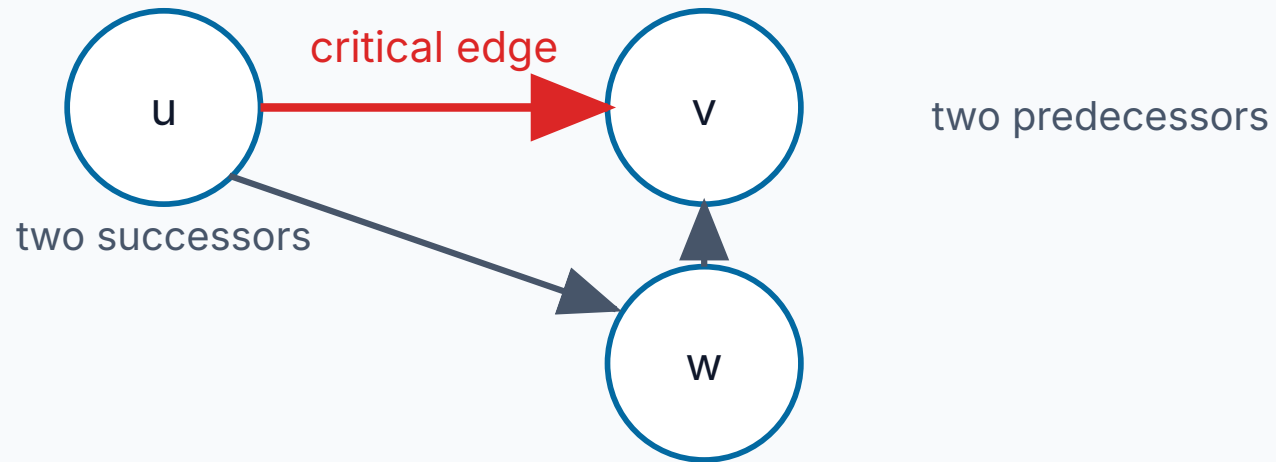
$$x = \text{ite}(bb_i, x_i, x_k)$$

After Boolean propagation learns bb_i , the ITE collapses to $x = x_i$.

Pitfall: Critical Edges

An edge $u \rightarrow v$ is critical when:

$$|\text{succ}(u)| > 1 \wedge |\text{pred}(v)| > 1$$



Block variables alone do not identify which incoming edge was taken.

Critical Edge Example

```
entry:
  c := havoc
  if c goto join else mid

mid:
  goto join

join:
  ok := phi [entry: true, mid: false]
```

Bad true-edge encoding:

$$(bb_{\text{entry}} \wedge bb_{\text{join}}) \Rightarrow c$$

On path entry -> mid -> join, both blocks are selected while c is false. The real bug path can be incorrectly rejected.

Pitfall: ITE Merge Exclusivity

If both predecessors can be selected, independent ITE merges can create a mixed state:

$$\begin{aligned}x &= \text{ite}(\text{bb}_{\text{left}}, x_{\text{left}}, x_{\text{right}}) \\p &= \text{ite}(\text{bb}_{\text{right}}, p_{\text{right}}, p_{\text{left}})\end{aligned}$$

Repair:

$$\neg(\text{bb}_{\text{left}} \wedge \text{bb}_{\text{right}})$$

At most one incoming predecessor may feed a merge.

VCGen

Boogie Style

Idea

Boogie-style VCGen uses backward safety equations over the CFG.

For each block i , introduce:

$$ok_i$$

ok_i means: starting execution at block i is safe.

The counterexample query is:

$$\neg ok_{\text{entry}}$$

Block Equation

$$\text{ok}_i \Leftrightarrow \left(\text{defs}_i \Rightarrow \left(\text{asserts}_i \wedge \bigwedge_{j \in \text{succ}(i)} \left((\text{cond}_{i,j} \wedge \text{dsa}_{i,j}) \Rightarrow \text{ok}_j \right) \right) \right)$$

- assumptions and definitions are premises
- assertions are consequents
- each feasible successor must be safe
- infeasible blocks are safe vacuously

Terminal Block

For a terminal block, the successor conjunction is true:

$$ok_i \Leftrightarrow (defs_i \Rightarrow asserts_i)$$

A false reachable assertion makes the local `ok_i` false, then unsafety propagates backward to entry.

DSA Diamond

entry:

x := havoc

y := havoc

c := x < y

if c goto left else right

left:

a_left := x + 1

a := a_left

goto join

right:

a_right := y + 1

a := a_right

goto join

DSA Diamond: Exit

```
join:
  ok := a > 0
  goto exit

exit:
  assume true
  assert ok
  halt
```

The assignments to `a` are edge-sensitive DSA definitions for the incoming edges to `join`.

Encoded Equations

$$\mathbf{OK}_{\text{entry}} : \text{ok}_{\text{entry}} \Leftrightarrow (c = (x < y) \Rightarrow ((c \Rightarrow \text{ok}_{\text{left}}) \wedge (\neg c \Rightarrow \text{ok}_{\text{right}})))$$

$$\mathbf{OK}_{\text{left}} : \text{ok}_{\text{left}} \Leftrightarrow (a_{\text{left}} = x + 1 \Rightarrow ((a = a_{\text{left}}) \Rightarrow \text{ok}_{\text{join}}))$$

$$\mathbf{OK}_{\text{right}} : \text{ok}_{\text{right}} \Leftrightarrow (a_{\text{right}} = y + 1 \Rightarrow ((a = a_{\text{right}}) \Rightarrow \text{ok}_{\text{join}}))$$

Encoded Exit

$$\mathbf{OK}_{\text{join}} : \text{ok}_{\text{join}} \Leftrightarrow (\text{ok} = (a > 0) \Rightarrow \text{ok}_{\text{exit}})$$

$$\mathbf{OK}_{\text{exit}} : \text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$$

$$\mathbf{DISCHARGE} : \neg \text{ok}_{\text{entry}}$$

The model chooses havoc values and a branch whose successor is unsafe.

Static ITE Merge Variant

The merge can be compiled into an ordinary definition:

```
join:  
  a := ite(c, a_left, a_right)  
  ok := a > 0  
  goto exit
```

$$\text{ok}_{\text{join}} \Leftrightarrow ((a = \text{ite}(c, a_{\text{left}}, a_{\text{right}}) \wedge \text{ok} = (a > 0)) \Rightarrow \text{ok}_{\text{exit}})$$

Tradeoffs

Advantages:

- no separate path-selection formula
- many assertions are natural
- assumptions are handled by implication

Costs:

- formula is nested
- cross-block simplification is harder
- phi/ITE extensions need exclusivity side conditions

Phi Extension Needs CFG Facts

If a phi is compiled as:

$$a = \text{ite}(\text{bb}_{\text{left}}, a_{\text{left}}, a_{\text{right}})$$

then the block variables used by the ITE must be consistent:

$$\begin{aligned} \text{bb}_{\text{join}} &\Rightarrow \text{bb}_{\text{left}} \vee \text{bb}_{\text{right}} \\ &\neg(\text{bb}_{\text{left}} \wedge \text{bb}_{\text{right}}) \end{aligned}$$

The backward equations prove safety; the CFG facts keep the ITE merge faithful.

VCGen

SeaBMC Style

Idea

SeaBMC-style VCGen compiles control-dependence into data-dependence.

After preprocessing:

- no block variables are needed
- no separate CFG constraints are emitted
- phi nodes become ordinary `ite` definitions
- cone-of-influence reduction is just data-flow slicing

Gated SSA

A gamma node is a value-level merge:

$$x := \gamma(g, x_t, x_f)$$

It means:

$$x := \text{ite}(g, x_t, x_f)$$

Unlike a phi node, gamma mentions a Boolean program expression, not a predecessor block name.

SSA Diamond

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto left else right

left:
  a_left := x + 1
  goto join

right:
  a_right := y + 1
  goto join

join:
  a := phi [left: a_left, right: a_right]
```

GSSA Diamond

The branch condition c selects the incoming region.

```
join:  
  a := gamma(c, a_left, a_right)  
  ok := a > 0  
  goto exit
```

After desugaring:

```
join:  
  a := ite(c, a_left, a_right)  
  ok := a > 0  
  goto exit
```

Flat VC

After phi elimination:

$$\text{VCGen_SeaBMC}(P) = \text{DEF} \wedge \text{ASSUME} \wedge \neg \text{ASSERT}$$

For the diamond:

$$\text{DEF} = c = (x < y) \wedge a_{\text{left}} = x + 1 \wedge a_{\text{right}} = y + 1 \wedge a = \text{ite}(c, a_{\text{left}}, a_{\text{right}}) \wedge \text{ok} = (a > 0)$$

Gate Construction

For each block b , compute a Boolean expression $\text{gate}(b)$:

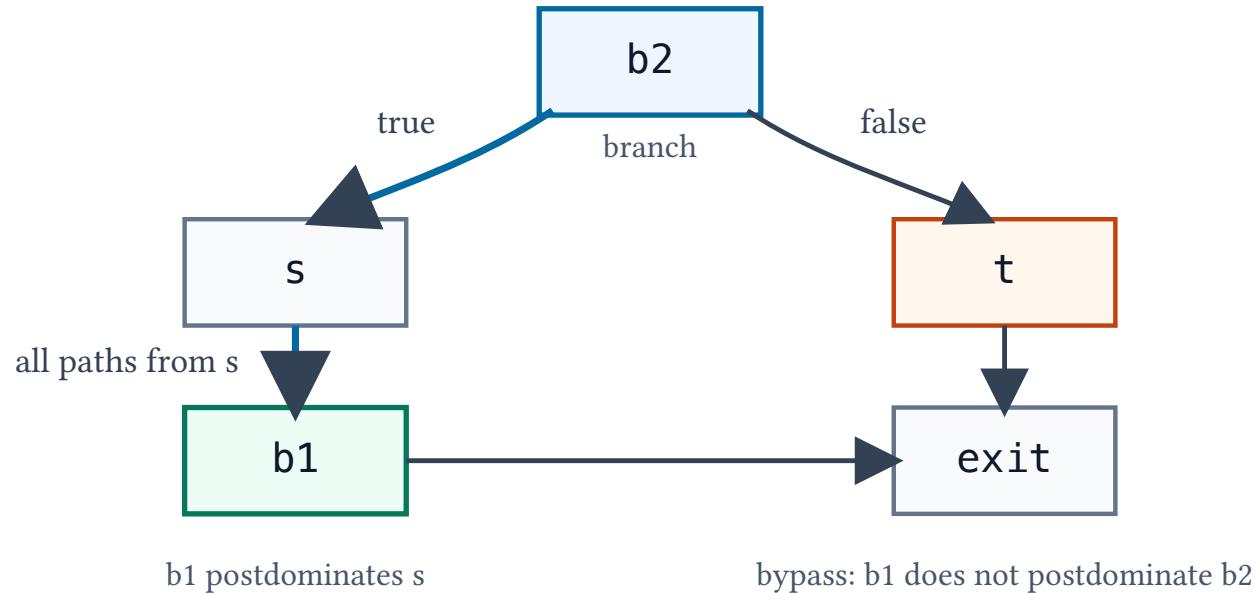
$$\text{gate}(\text{entry}) = \top$$

$$\text{gate}(b) = \bigvee_{c \in \text{ctrl}(b)} (\text{gate}(c) \wedge \text{orient}(c, b))$$

$\text{ctrl}(b)$ contains controlling branch blocks. $\text{orient}(c, b)$ is the branch condition or its negation, depending on which successor reaches b .

Control Dependence

Control dependence: b1 depends on the branch at b2



A block is control-dependent on a branch when one successor forces a later visit to the block, while another successor can bypass it.

Materialized Gates

Expanding every gate at every use can duplicate shared control structure.

Materialized gates introduce ordinary SSA Boolean definitions:

$$\begin{aligned} G_{\text{entry}} &:= \top \\ G_b &:= \bigvee_{c \in \text{ctrl}(b)} (G_c \wedge \text{orient}(c, b)) \\ x &:= \gamma(G_p, v_p, v_q) \end{aligned}$$

The VC remains CFG-free; gates are just definitions.

Shared Gate Example

```
gate_a := p
gate_b := q
gate_c := (gate_a and r) or (gate_b and s)
gate_d := (gate_b and t) or (gate_c and u)
gate_e := (gate_c and v) or (gate_d and w)

join1:
  x := ite(gate_d, x_d, ite(gate_e, x_e, x_other))

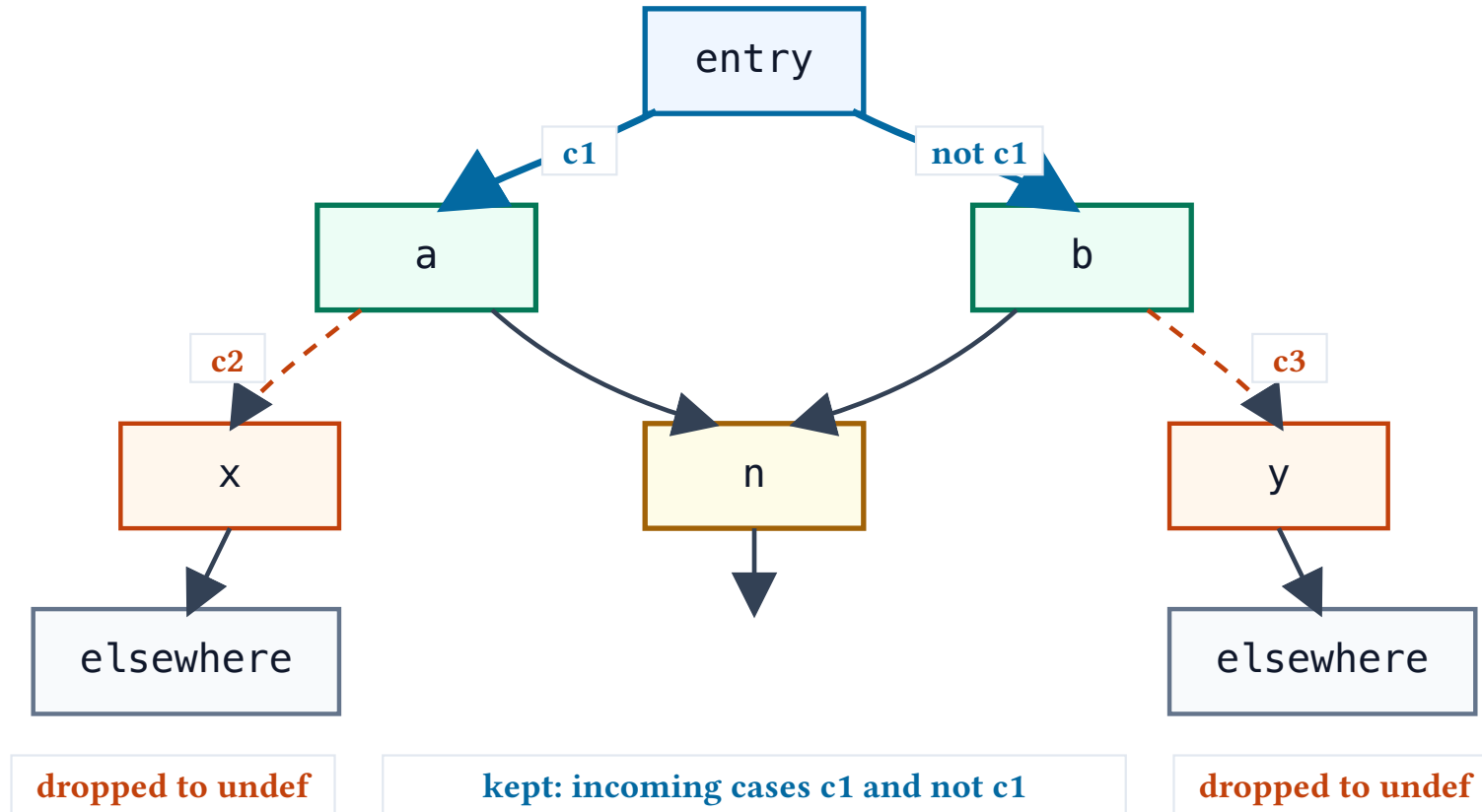
join2:
  y := ite(gate_d, y_d, ite(gate_e, y_e, y_other))
```

The shared control-dependence DAG is represented once.

Thin GSSA

Thin GSSA uses only direct control-dependence controllers.

Thin GSSA keeps the incoming cases to n and drops bypass cases to undef



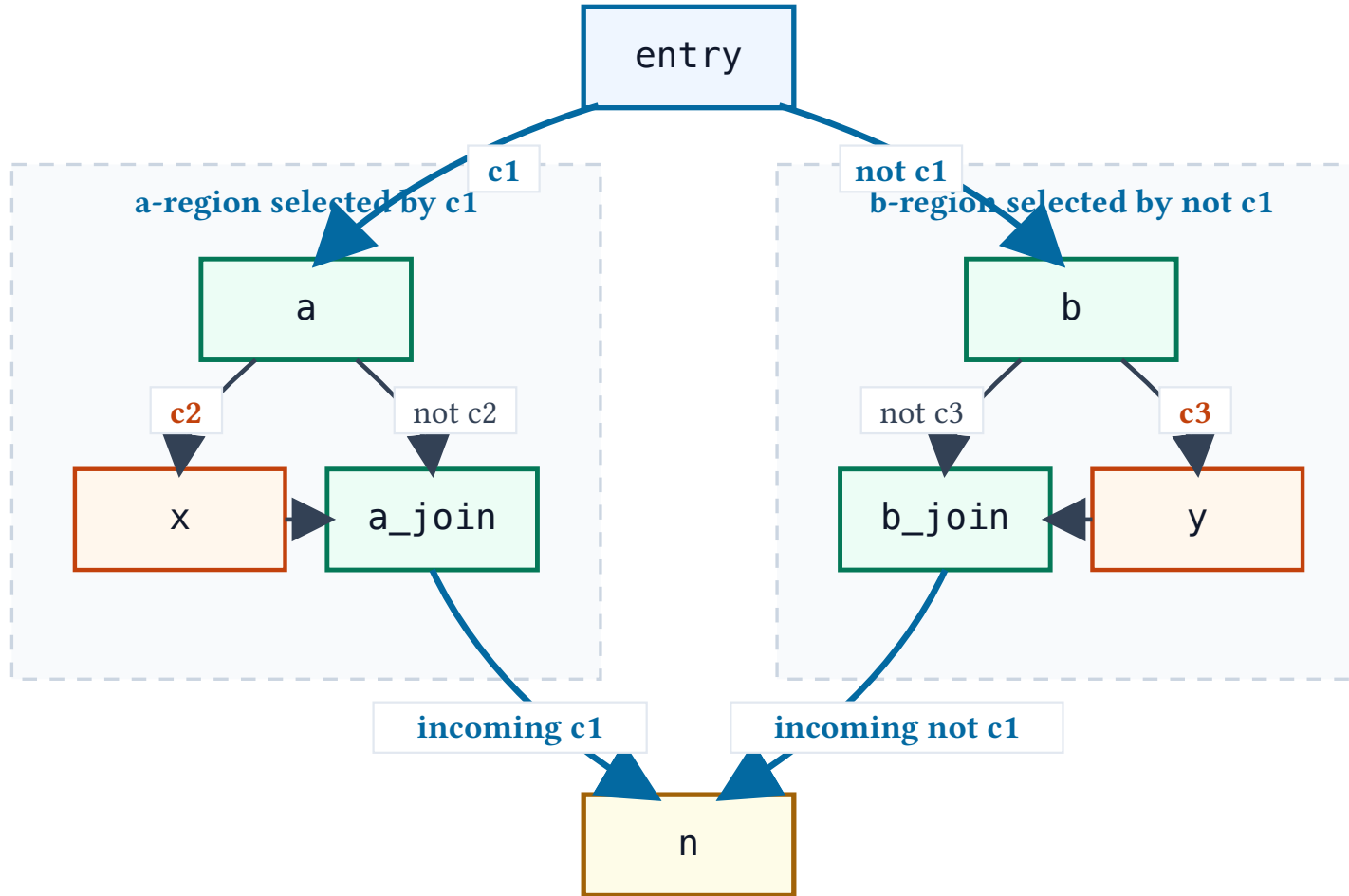
For a direct controller c of block b :

$$K_{c,b} = G_c \wedge \text{orient}(c, b)$$

The G_c factor prevents unreachable branch conditions from selecting values.

Thin Example CFG

CFG for the Thin GSSA example



The outer branch selects the region feeding n ; local branches only compute values inside each region.

Thin Example: Final Form

```
a_join:
  v_a := ite(c2, v_x, v_a0)

b_join:
  v_b := ite(c3, v_y, v_b0)

n:
  v := ite(c1, v_a, v_b)
```

The final phi at n depends only on the region selector c_1 , not on the local branch choices c_2 and c_3 .

Cone Of Influence

Once control is data:

1. Start from the exit assume and assert.
2. Keep each used variable definition.
3. Add the variables used by that definition.
4. Keep gates only when retained ITEs use them.
5. Drop every unmarked definition.

No graph reachability rule is needed for pruning.

COI Example

```
n:  
  v := ite(c1, v_a, v_b)  
  junk := expensive(v_x, v_y)  
  ok := v > 0  
  
exit:  
  assume true  
  assert ok
```

junk is dropped because it is not used by `ok`, the exit assumption, or a retained gate.

VC Generation

References And Borrowing

Goal

Extend Tiny TAC with references without adding a new VCGen core.

Plan:

- model a reference as an explicit value
- lower borrow commands to ordinary assignments, map reads, map writes, and assumes
- run the existing Tiny TAC VCGen

Reference Types

Tiny TAC gains:

$$\tau ::= \text{bool} \mid \text{int} \mid \text{bytemap} \mid \& \tau \mid \&\text{mut } \tau$$

The main case is a reference to an integer cell:

$$r : \&\text{int} \quad q : \&\text{mut int}$$

Reference Triple

Symbolically, a reference carries:

$$r = \{\text{addr} : i, \text{value} : v, \text{promise} : p\}$$

- `addr`: location pointed to by the reference
- `value`: current value observed through the reference
- `promise`: value that will be committed on release

The `promise` field is unused for constant references, but keeping one shape makes the lowering uniform.

Borrow Commands

```
r := borrow M[i]  
r, M2 := borrow_mut M[i]  
q := borrow_ref r  
q, r2 := borrow_ref_mut r  
x := get_ref r  
r2 := put_ref r, v  
release r
```

Mutable borrowing returns both the reference and a continuation memory. Mutable reborrowing returns the child reference and the resumed parent reference.

Direct Borrow Example

```
entry:
  M := havoc
  i := havoc
  p := borrow M[i]
  x := get_ref p
  release p
  q, M2 := borrow_mut M[i]
  q2 := put_ref q, (x + 1)
  release q2
  ok := M2[i] == x + 1
  assert ok
```

This reads through a constant reference, then updates the same location through a mutable reference.

Lowering: Direct Borrows

Constant borrow:

```
r := borrow M[i]
```

lowers to:

```
r := { addr: i,  
       value: M[i],  
       promise: havoc }
```

Mutable borrow:

```
r, M2 := borrow_mut M[i]
```

lowers to:

```
r := { addr: i,  
       value: M[i],  
       promise: havoc }  
M2 := M[i := r.promise]
```

Lowering: Use And Release

Read:

```
x := get_ref r
```

lowers to:

```
x := r.value
```

Write:

```
r2 := put_ref r, v
```

lowers to:

```
r2 := { addr: r.addr,  
        value: v,  
        promise: r.promise }
```

Release:

```
assume r.value == r.promise
```

Mutable Borrow Lowered

```
entry:
  M := havoc
  i := havoc
  r := { addr: i, value: M[i], promise: havoc }
  M2 := M[i := r.promise]
  r2 := { addr: r.addr, value: 7, promise: r.promise }
  assume r2.value == r2.promise
  x := M2[i]
  ok := x == 7
  assert ok
```

Why It Proves

$$\begin{aligned}M2 &= M[i := \text{promise}(r)] \\ \text{value}(r2) &= 7 \\ \text{promise}(r2) &= \text{promise}(r) \\ \text{value}(r2) &= \text{promise}(r2)\end{aligned}$$

Therefore:

$$\text{promise}(r) = 7$$

So the continuation memory $M2$ stores 7 at address i .

Mutable Reborrow

```
r, M2 := borrow_mut M[i]
q, r2 := borrow_ref_mut r
q2 := put_ref q, 7
release q2
r3 := put_ref r2, 8
release r3
```

`q` borrows through `r`. After `q` is released, `r2` resumes the parent reference and can overwrite the final promised value.

Reborrow Lowering

```
q := { addr: r.addr, value: r.value, promise: havoc }  
r2 := { addr: r.addr, value: q.promise, promise: r.promise }
```

When `q` is released, it updates `r2.value`. When `r2` is released, it updates the original memory promise.

Reborrow Facts

$$\begin{aligned}M2 &= M[i := \text{promise}(r)] \\ \text{value}(q2) &= 7 \\ \text{value}(q2) &= \text{promise}(q) \\ \text{value}(r3) &= 8 \\ \text{promise}(r3) &= \text{promise}(r2) \\ \text{promise}(r2) &= \text{promise}(r)\end{aligned}$$

The final release implies $\text{promise}(r) = 8$, so $M2[i] == 8$.

Prophecy Variable

The `promise` field is a prophecy-style value:

- chosen nondeterministically at borrow time
- constrained later at release time
- already written into the continuation memory

This is the RustHorn idea adapted to a low-level memory setting: future borrow effects are represented by values in the current formula.

VCGen Boundary

After lowering, VCGen only sees ordinary Tiny TAC:

- assignments
- map reads and stores
- assumptions
- assertions
- havoc values

Borrow well-formedness remains a separate side condition or a separate set of verification obligations.